Business Intelligence

Proyecto final · Entrega 2

Data warehouse, ETL y tablero

RetailBI

Integrante	Código
Juan David Pardo Gutierrez	506241031
Linda Valentina Quintero Quiroga	506241056
Daniel Felipe Rodríguez Angulo	506241129

Dato	Valor
Docente	Leandro Pájaro Fuentes
Fecha de entrega	06/11/2026
Repositorio	https://github.com/damoelrrr/BI-Entrega-2-Equipo.506241031-506241056-506241129
Motor de base de datos	PostgreSQL 16.15 (local)
Herramienta del tablero	Power BI Desktop

Resumen de la entrega

En esta entrega se implementó en PostgreSQL el modelo dimensional en estrella diseñado en la Entrega 1: cuatro dimensiones (dim_fecha, dim_tienda, dim_producto y dim_descuento), la tabla de hechos fact_ventas y dos tablas de contexto para F1 y F2, es decir, siete tablas. Un trabajo de Pentaho Data Integration (job_carga_retailbi.kjb) ejecuta seis transformaciones, limpia los problemas de calidad diagnosticados y carga 49.899 registros en fact_ventas (101 de las 50.000 líneas de F3 se rechazaron por cantidad vacía) a partir de los datos sintéticos de F3. Las cifras demuestran el funcionamiento del modelo y no corresponden a ventas reales de una empresa.

Respuesta a la pregunta principal (P0): entre 2023 y 2025 los ingresos netos sumaron $1.211.283.800 COP con 124.218 unidades vendidas, y crecieron de $377,4 millones en 2023 a $431,0 millones en 2025 (+14,2 %). Kennedy lidera en unidades y Carnes y huevos en ingresos netos.

1. Modelo físico

1.1 Motor y base de datos

Motor: PostgreSQL 16.15, instalado localmente en el equipo de Juan David Pardo Gutierrez (Windows, puerto 5432). Base de datos: retailbi_dw. Esquema: dw, que contiene las tablas del modelo y las de contexto.

Frente a la Entrega 1 no se creó un esquema staging: la capa es opcional en esta entrega, por lo que PDI limpia y valida los datos en memoria, carga directamente en dw y escribe las líneas rechazadas en un archivo de rechazos (sección 3).

1.2 Tablas creadas

Las llaves de las dimensiones son subrogadas (las genera la base al cargar); la llave de la fuente (tienda_id, producto_id) se conserva como atributo único. dim_fecha usa la llave AAAAMMDD. dim_tienda, dim_producto y dim_descuento incluyen una fila con llave −1 «Desconocido» para los hechos sin coincidencia; por eso tienen 6, 31 y 5 registros (5 tiendas, 30 productos y 4 rangos de descuento, más esa fila).

Tabla	Tipo	Llave primaria	Llaves foráneas	Registros	Script DDL
dim_fecha	Dimensión	fecha_key	—	1.096	dw/ddl/02_dim_fecha.sql
dim_tienda	Dimensión	tienda_key	—	6	dw/ddl/03_dim_tienda.sql
dim_producto	Dimensión	producto_key	—	31	dw/ddl/04_dim_producto.sql
dim_descuento	Dimensión	descuento_key	—	5	dw/ddl/05_dim_descuento.sql
fact_ventas	Hechos	venta_fact_key	fecha_key, tienda_key, producto_key, descuento_key	49.899	dw/ddl/06_fact_ventas.sql
ctx_emc_comercio_minorista	Contexto (F1)	emc_id	—	92	dw/ddl/07_ctx_emc_comercio_minorista.sql
ctx_empresas_bogota	Contexto (F2)	empresa_id	—	39.941	dw/ddl/08_ctx_empresas_bogota.sql

1.3 Diagrama del modelo físico

El esquema implementado es una estrella: fact_ventas se relaciona directamente con cuatro dimensiones, sin tablas intermedias. Las tablas ctx_* no se relacionan con la estrella; sirven de contraste. El diagrama se generó

desde la base de datos con la herramienta ERD de pgAdmin 4, conectada a retailbi_dw y está en docs/diagramas/modelo_fisico.png.

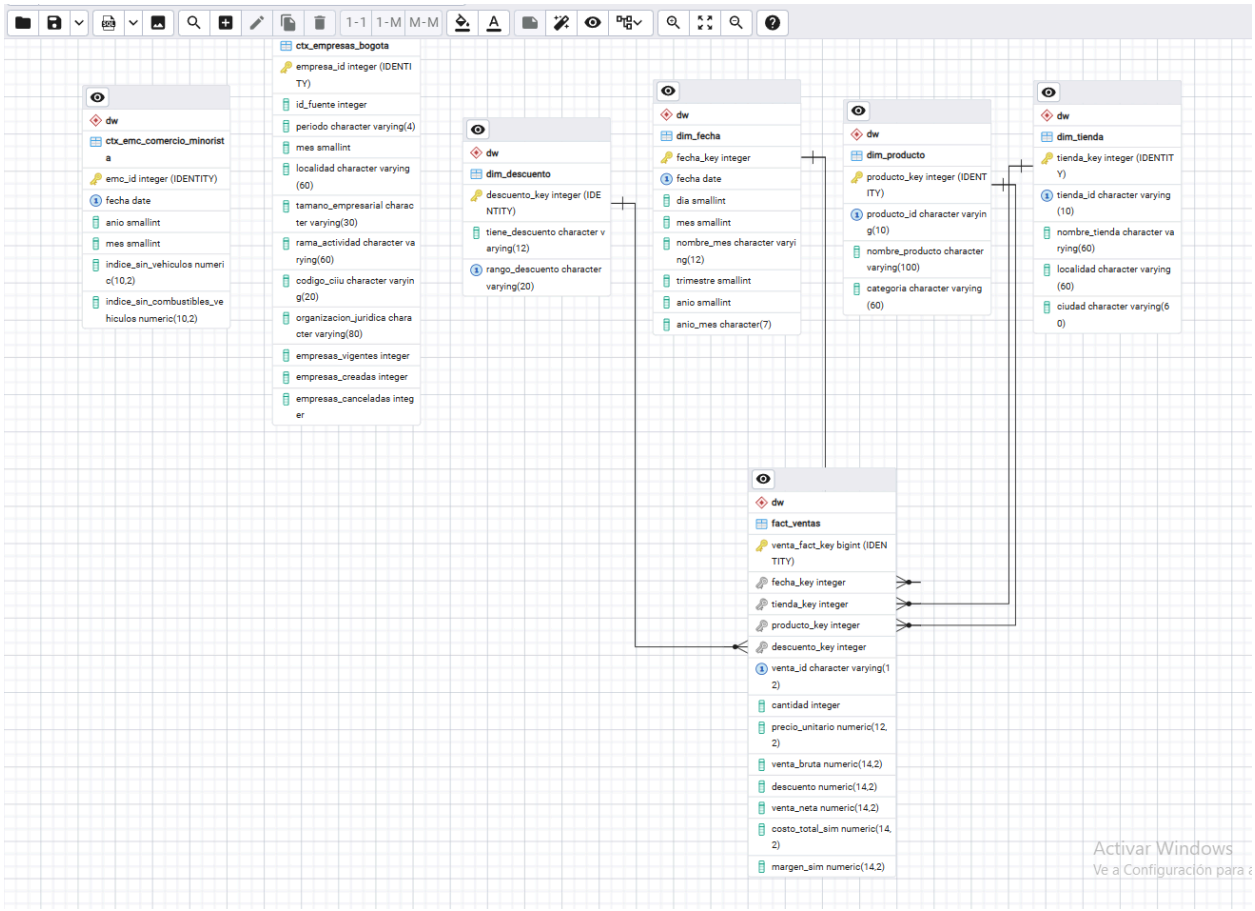


Figura 1. Diagrama del modelo físico generado con la herramienta ERD de pgAdmin 4 desde retailbi_dw (esquema en estrella).

1.4 Dimensión de tiempo

La dimensión dim_fecha cubre del 01/01/2023 al 31/12/2025, el periodo de los hechos de F3, con 1.096 registros: un día por registro, sin huecos (2024 es bisiesto). Se pobló con el script SQL dw/ddl/02_dim_fecha.sql, que usa generate_series. Sus atributos son fecha_key, fecha, dia, mes, nombre_mes, trimestre, anio y anio_mes, con la jerarquía día → mes → trimestre → año. La consulta 03_integridad_fact_ventas.sql comprueba que no faltan días y que todos los hechos tienen fecha.

2. Procesos ETL

2.1 Trabajo de carga

El trabajo job_carga_retailbi.kjb (etl/trabajos/) ejecuta, en este orden: START → t_dim_tienda → t_dim_producto → t_dim_descuento → t_ctx_emc → t_ctx_empresas → t_fact_ventas → Éxito. Las dimensiones se cargan antes que los hechos porque fact_ventas necesita sus llaves. Si una transformación falla, el trabajo termina con «Abortar». dim_fecha se pobló antes con el script SQL (sección 1.4).

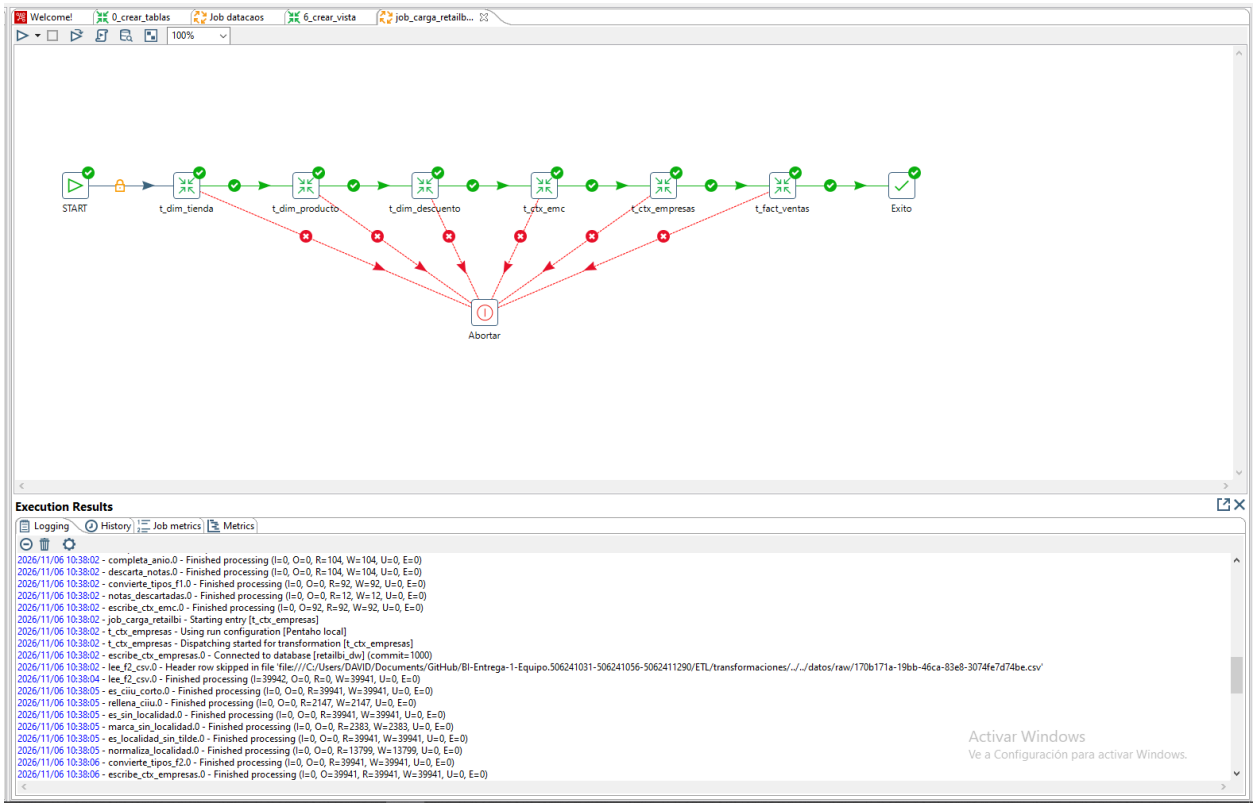


Figura 2. Trabajo job_carga_retailbi.kjb en Spoon (PDI 9.4): las seis transformaciones en orden, con salida a Éxito y a Abortar si alguna falla.

2.2 Transformaciones

Las transformaciones leen las fuentes en su formato original: F1 en XLSX, F2 en CSV (tal como se descargó del portal) y F3 en CSV (copia con inconsistencias de la fuente original). Los leídos, escritos y rechazos salen de la pestaña de métricas de pasos de la ejecución.

Archivo .ktr	Fuente(s) y formato	Destino	Pasos principales	Leídos	Escritos	Rechazos
t_dim_tienda.ktr	F3 (CSV)	dim_tienda	CSV file input → limpia_espacios → Sort rows → Unique rows → Insert/Update	50.000	5	0
t_dim_producto.ktr	F3 (CSV)	dim_producto	CSV file input → limpia_espacios → Sort rows → Unique rows → Insert/Update	50.000	30	0
t_dim_descuento.ktr	Rangos definidos en la Entrega 1 (Data grid)	dim_descuento	Data grid → Insert/Update	4	4	0

Archivo .ktr	Fuente(s) y formato	Destino	Pasos principales	Leídos	Escritos	Rechazos
t_ctx_emc.ktr	F1 (XLSX)	ctx_emc_comercio_minorista	Microsoft Excel input → limpia_mes → mes_a_numero → completa_anio → descarta_notas → Table output	104	92	12
t_ctx_empresas.ktr	F2 (CSV)	ctx_empresas_bogota	CSV file input → es_ciiu_corto → rellena_ciiu → es_sin_localidad → marca_sin_localidad → es_localidad_sin_tilde → normaliza_localidad → convierte_tipos_f2 → Table output	39.941	39.941	0
t_fact_ventas.ktr	F3 (CSV)	fact_ventas	CSV file input → limpia_espacios → normaliza_fecha → rechaza_nulos → convierte_tipos → quita_duplicados → calcula_metricas (Calculator) → valida_total → calcula_rango → Database lookup (tienda, producto, descuento) → Table output	50.000	49.899	101

2.3 Reglas de limpieza aplicadas

Cada regla tiene un paso de PDI con nombre propio. El número de registros afectados se lee de las métricas del paso que aplica la regla (un Filter rows le envía solo las filas afectadas) o, en los pasos Modified JavaScript, del contador que el paso escribe en el log de la ejecución y se contrasta con el log de inconsistencias de F3 (datos/inconsistencias/).

Problema (Entrega 1)	Fuente y campo	Regla aplicada	Paso de PDI (nombre)	Registros afectados
Mes como texto con espacios y punto sobrante	F1.Mes	Quitar espacios y punto; convertir el nombre del mes a número	limpia_mes (Modified JavaScript) y mes_a_numero (Value mapper)	92
Año solo en la primera fila de cada año	F1.Año	Arrastrar hacia abajo el último año no vacío	completa_anio (Modified JavaScript)	84
Filas de notas debajo de los datos	F1 (filas finales)	Descartar filas sin año o mes válido, como «Fuente: DANE - EMC»	descarta_notas (Filter rows)	12
Códigos CIIU sin cero inicial	F2.Codigo CIIU Rev. 4 A.C	Rellenar con ceros a la izquierda hasta cuatro dígitos	rellena_ciiu (String operations, Pad left)	2.147

Problema (Entrega 1)	Fuente y campo	Regla aplicada	Paso de PDI (nombre)	Registros afectados
Localidad «Sin Localidad»	F2.Localidad	Conservar el registro y unificar el valor como «Sin localidad»	marca_sin_localidad (Value mapper)	2.383
Nombres de localidad sin tildes	F2.Localidad	Reemplazar por el nombre oficial (Engativa → Engativá, Usaquen → Usaquén, Fontibon → Fontibón, Candelaria → La Candelaria, etc.)	normaliza_localidad (Value mapper)	13.799
Fechas en formatos distintos	F3.fecha_venta	Convertir aaaa-mm-dd, dd/mm/aaaa y mm-dd-aaaa a fecha; el separador y la posición del año identifican el formato	normaliza_fecha (Modified JavaScript)	1.502
Espacios sobrantes en nombres	F3.tienda y producto	Recortar espacios al inicio y al final	limpia_espacios (String operations)	501
Nulos en campos obligatorios	F3.cantidad y otros	Rechazar la línea y escribirla en el archivo de rechazos	rechaza_nulos (Filter rows)	101
Duplicados de venta_id (validación de la Entrega 1, tabla 4.2)	F3.venta_id	Conservar una sola fila por venta_id	quita_duplicados (Unique rows)	0
Total distinto de cantidad × precio − descuento (Entrega 1, tabla 4.2)	F3.total_venta_cop	Rechazar la línea que no cuadra	valida_total (Filter rows)	0

2.4 Evidencia de ejecución

La captura muestra la ejecución completa de job_carga_retailbi.kjb sobre una base vacía, terminada sin errores, con la pestaña de métricas de pasos visible. Fecha y hora de la ejecución: 06/11/2026, de 10:37:59 a 10:38:09. Duración: 10 segundos.

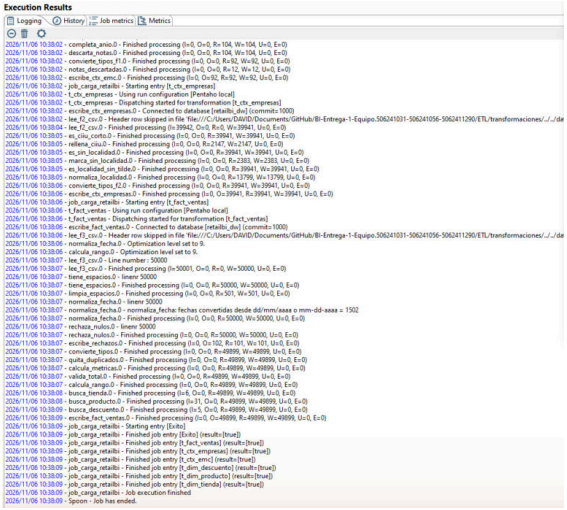


Figura 3. Log de la ejecución completa de job_carga_retailbi.kjb en Spoon, con las métricas de lectura y escritura de cada paso.

3. Conciliación

3.1 Conteo por fuente

Leídos = cargados + descartados. Los leídos y descartados salen de las métricas de PDI y los cargados, de la consulta 01_conteo_tablas_dw.sql (Anexo A).

Fuente	Leídos	Cargados	Descartados	Motivo de los descartes
F1	104 (92 meses + 12 filas de notas)	92	12	Filas de notas bajo la tabla (descarta_notas).
F2	39.941	39.941	0	Ninguno: todas las filas se cargan como contexto (las correcciones no descartan registros).
F3	50.000	49.899	101	Cantidad vacía (nulo en campo obligatorio): 101 líneas, registradas en datos/rechazos/f3_rechazos.csv. No hubo duplicados ni totales inconsistentes.

3.2 Totales de una métrica

El total de F3 se calcula sobre el CSV original de datos/raw/ (consulta 04, DuckDB), no sobre la copia con inconsistencias; así la diferencia equivale a lo que la limpieza rechazó.

Métrica	Total en la fuente	Total en el DW	Diferencia y explicación
Líneas de venta	50.000	49.899	101 líneas rechazadas por cantidad vacía
Unidades vendidas	124.460	124.218	242 unidades: las de las 101 líneas rechazadas
Venta neta (COP)	1.213.787.500	1.211.283.800	2.503.700 COP: suma de total_venta_cop de las 101 líneas rechazadas

4. Tablero

4.1 Indicadores

Los indicadores se muestran frente a una referencia (mes anterior o promedio) en tarjetas de la página 1. Las medidas están definidas en DAX sobre fact_ventas y dim_fecha.

Indicador	Fórmula (DAX o SQL)	Qué mide	Referencia	Pregunta
Venta neta	`Venta neta = SUM(fact_ventas[venta_neta])`	Ingresos tras descontar descuentos (COP)	Mes anterior: `CALCULATE([Venta neta], DATEADD(dim_fecha[fecha], -1, MONTH))`	P0, P1

Indicador	Fórmula (DAX o SQL)	Qué mide	Referencia	Pregunta
Unidades vendidas	`Unidades = SUM(fact_ventas[cantidad])`	Volumen vendido	Promedio mensual: `AVERAGEX(ALLSELECTED(dim_fecha[anio_mes]), [Unidades])`	P0, P2
% de descuento	`% Descuento = DIVIDE(SUM(fact_ventas[descuento]), SUM(fact_ventas[venta_bruta]))`	Parte de la venta bruta cedida en descuentos	Promedio general: `CALCULATE([% Descuento], ALL(dim_fecha), ALL(dim_tienda), ALL(dim_producto))`	P1
Variación mensual de venta neta	`Var % = DIVIDE([Venta neta] - [Venta neta mes anterior], [Venta neta mes anterior])`	Aumento o caída frente al mes previo	Mes anterior (0 %)	P3

4.2 Respuesta a las preguntas de negocio

Se responden las cuatro preguntas de tipo BI (P0 a P3). P4 es de tipo BA y se aborda en una entrega posterior.

Pregunta	Página y visual	Tipo de gráfico y por qué	Respuesta encontrada (con cifra)
P0	Página 1 «Evolución»: líneas «Venta neta mensual por tienda (COP)» y tarjetas de indicadores (venta neta y unidades)	Líneas para la evolución en el tiempo, con una serie por tienda; columnas para las unidades, que son un volumen por periodo	Los ingresos netos crecieron de $377,4 millones (2023) a $402,9 millones (2024) y $431,0 millones (2025); las unidades, de 38.839 a 44.020.
P1	Página 2 «Productos y tiendas»: barras horizontales «Top 10 productos por venta neta (COP)» y barras «Venta neta por categoría»	Barras ordenadas porque comparan magnitudes entre categorías discretas; se evita la torta por el número de categorías	Carnes y huevos lidera con $288,3 millones (23,8 % de la venta neta) y Granos es la menor con $97,2 millones; el producto líder es Papel higiénico x12 con $91,2 millones.
P2	Página 2 «Productos y tiendas»: barras «Unidades vendidas por tienda»	Barras ordenadas para comparar las cinco tiendas en una misma magnitud	Kennedy vendió 29.919 unidades, 45,5 % más que Usaquén, la última con 20.567.
P3	Página 2 «Productos y tiendas»: columnas «Variación mensual de la venta neta (%)»	Columnas con color según signo: muestran de inmediato qué meses subieron o bajaron	Mayor alza en diciembre de 2023 (+33,6 %) y mayor caída en enero de 2024 (−34,1 %); 20 meses subieron y 15 bajaron frente al mes anterior.

4.3 Capturas del tablero

El tablero (dashboard/RetailBI_tablero.pbix) tiene dos páginas y se conecta a la base retailbi_dw de PostgreSQL con las tablas dw.fact_ventas y dw.dim_*; no usa los archivos de las fuentes. Las relaciones son de muchos a uno desde fact_ventas hacia cada dimensión por su llave, y dim_fecha está marcada como tabla de fechas. Las capturas están también en dashboard/.

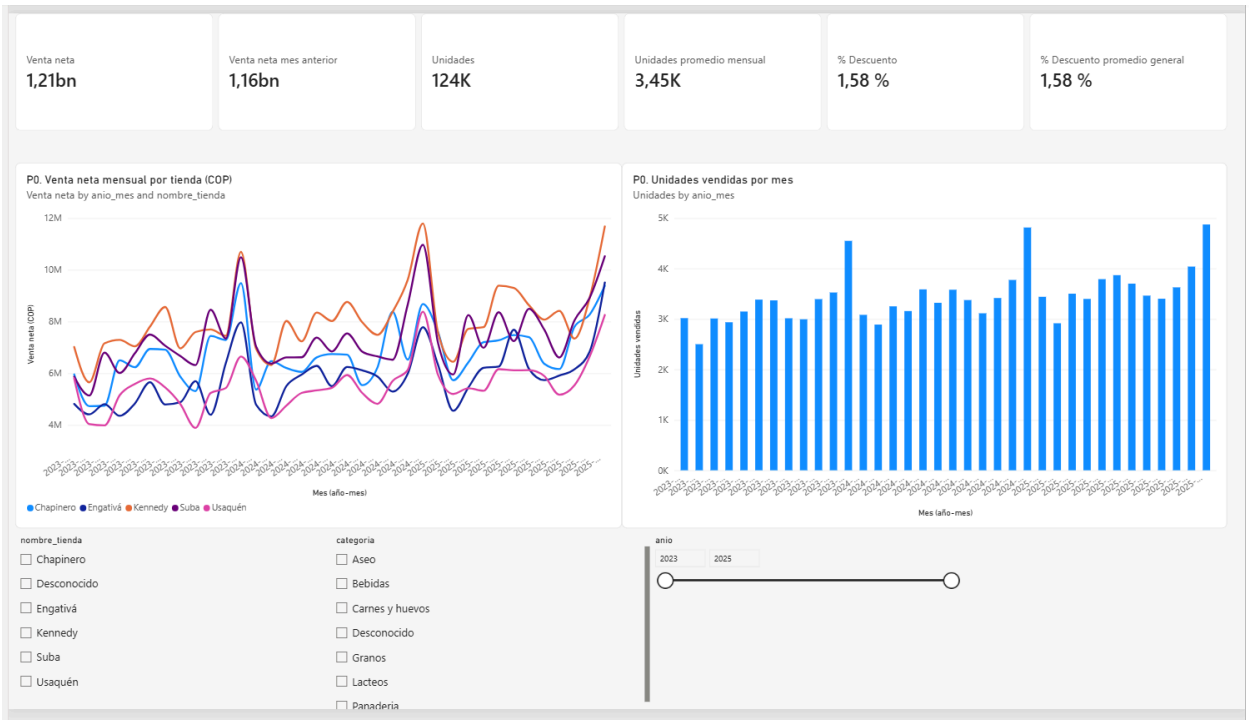


Figura 4. Tablero RetailBI, página 1: tarjetas de indicadores, venta neta mensual por tienda y unidades vendidas por mes (P0).

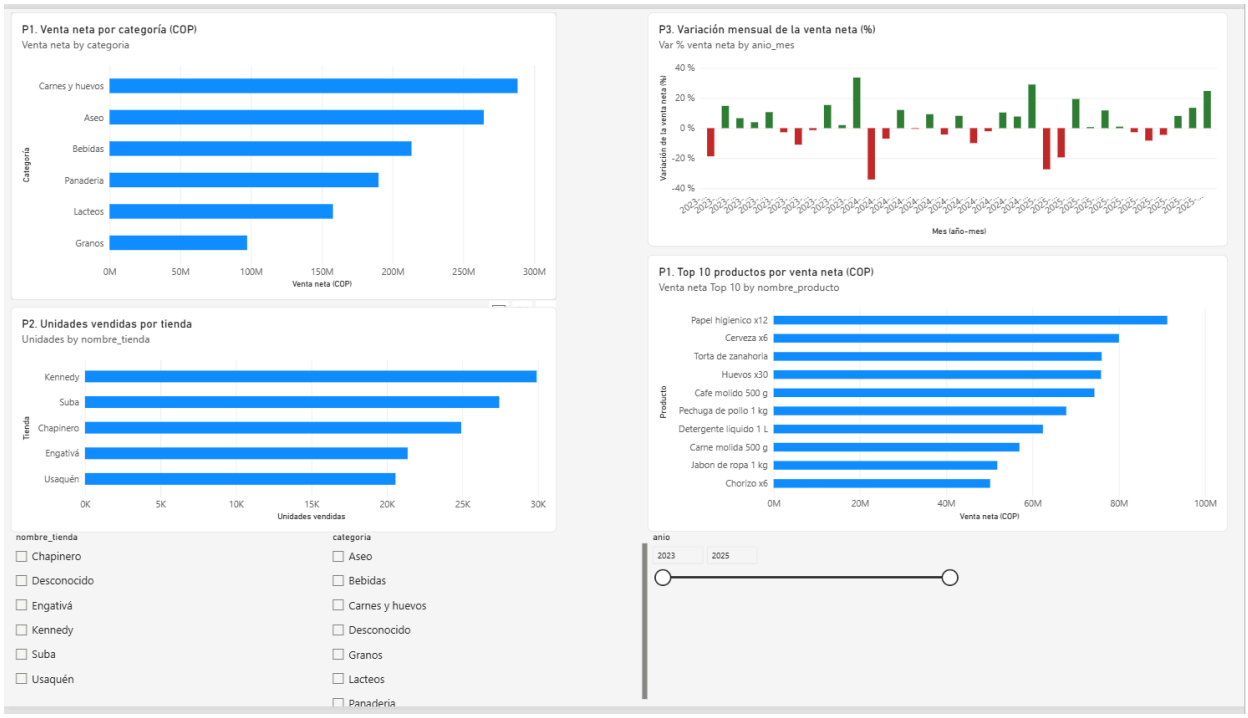


Figura 5. Tablero RetailBI, página 2: venta neta por categoría (P1), top 10 productos (P1), unidades por tienda (P2) y variación mensual (P3).

4.4 Filtros

Las dos páginas tienen los mismos segmentadores: Año (dim_fecha[anio], rango deslizante; dimensión de tiempo), Tienda (dim_tienda[nombre_tienda]) y Categoría (dim_producto[categoria]). Cada segmentador afecta a todos los visuales de su página. Las referencias de los indicadores («promedio general») ignoran los filtros a propósito, para que la comparación se mantenga.

Anexos

Anexo A. Consultas de conciliación

Consultas idénticas a las de dw/conciliacion/.

dw/conciliacion/01_conteo_tablas_dw.sql

```
-- Registros cargados en cada tabla del data warehouse
SELECT 'dim_fecha'                 AS tabla, COUNT(*) AS registros FROM dw.dim_fecha
UNION ALL SELECT 'dim_tienda',               COUNT(*) FROM dw.dim_tienda
UNION ALL SELECT 'dim_producto',             COUNT(*) FROM dw.dim_producto
UNION ALL SELECT 'dim_descuento',            COUNT(*) FROM dw.dim_descuento
UNION ALL SELECT 'fact_ventas',              COUNT(*) FROM dw.fact_ventas
UNION ALL SELECT 'ctx_emc_comercio_minorista',COUNT(*) FROM dw.ctx_emc_comercio_minorista
UNION ALL SELECT 'ctx_empresas_bogota',      COUNT(*) FROM dw.ctx_empresas_bogota
ORDER BY tabla;
```

dw/conciliacion/02_totales_fact_ventas.sql

```
-- Totales de la tabla de hechos (comparar con la fuente F3, consulta 04)
SELECT COUNT(*)           AS lineas,
       SUM(cantidad)      AS unidades,
       SUM(venta_bruta)   AS venta_bruta,
       SUM(descuento)     AS descuento,
       SUM(venta_neta)    AS venta_neta
FROM dw.fact_ventas;
```

dw/conciliacion/03_integridad_fact_ventas.sql

```
-- Todas las consultas deben devolver 0
SELECT 'hechos sin fecha'      AS control, COUNT(*) AS filas
FROM dw.fact_ventas f LEFT JOIN dw.dim_fecha d ON d.fecha_key = f.fecha_key
WHERE d.fecha_key IS NULL
UNION ALL
SELECT 'hechos con tienda desconocida', COUNT(*) FROM dw.fact_ventas WHERE tienda_key = -1
UNION ALL
SELECT 'hechos con producto desconocido', COUNT(*) FROM dw.fact_ventas WHERE producto_key = -1
UNION ALL
SELECT 'hechos con descuento desconocido', COUNT(*) FROM dw.fact_ventas WHERE descuento_key = -1
UNION ALL
SELECT 'venta_neta <> bruta - descuento', COUNT(*)
FROM dw.fact_ventas WHERE venta_neta <> venta_bruta - descuento
UNION ALL
SELECT 'venta_id duplicado', COUNT(*) FROM (
    SELECT venta_id FROM dw.fact_ventas GROUP BY venta_id HAVING COUNT(*) > 1) x
UNION ALL
SELECT 'dias sin registro en dim_fecha (2023-2025)', 1096 - COUNT(*)
FROM dw.dim_fecha WHERE fecha BETWEEN DATE '2023-01-01' AND DATE '2025-12-31';
```

dw/conciliacion/04_totales_fuente_f3_duckdb.sql

```
-- Lado "fuente": se ejecuta con DuckDB (https://duckdb.org) sobre el CSV original,
-- sin pasar por PostgreSQL. Ajustar la ruta al archivo de datos/raw/.
SELECT COUNT(*)                   AS lineas,
       SUM(cantidad)              AS unidades,
       SUM(cantidad * precio_unitario_cop)  AS venta_bruta,
       SUM(descuento_cop)         AS descuento,
       SUM(total_venta_cop)       AS venta_neta
FROM read_csv_auto('datos/raw/RetailBI_ventas_sinteticas_50000.csv');
```

dw/conciliacion/05_verificacion_preguntas_negocio.sql

```
-- Cifras de contraste para el tablero (P0 a P3). Deben coincidir con los visuales de Power BI.
-- P0: venta neta y unidades por año
SELECT d.anio, SUM(f.venta_neta) AS venta_neta, SUM(f.cantidad) AS unidades
FROM dw.fact_ventas f JOIN dw.dim_fecha d USING (fecha_key) GROUP BY d.anio ORDER BY d.anio;
-- P1: venta neta por categoría (y top 5 productos)
SELECT p.categoria, SUM(f.venta_neta) AS venta_neta
FROM dw.fact_ventas f JOIN dw.dim_producto p USING (producto_key) GROUP BY p.categoria ORDER BY 2 DESC;
```

```
SELECT p.nombre_producto, SUM(f.venta_neta) AS venta_neta
FROM dw.fact_ventas f JOIN dw.dim_producto p USING (producto_key) GROUP BY p.nombre_producto ORDER BY 2
DESC LIMIT 5;
-- P2: unidades por tienda
SELECT t.nombre_tienda, SUM(f.cantidad) AS unidades
FROM dw.fact_ventas f JOIN dw.dim_tienda t USING (tienda_key) GROUP BY t.nombre_tienda ORDER BY 2 DESC;
-- P3: variación mensual de la venta neta
WITH m AS (SELECT d.anio_mes, SUM(f.venta_neta) AS vn FROM dw.fact_ventas f JOIN dw.dim_fecha d USING
(fecha_key) GROUP BY d.anio_mes)
SELECT anio_mes, vn, ROUND(100.0 * (vn - LAG(vn) OVER (ORDER BY anio_mes)) / LAG(vn) OVER (ORDER BY
anio_mes), 1) AS var_pct
FROM m ORDER BY anio_mes;
```

Anexo B. Capturas adicionales

El trabajo de carga y la ejecución completa en PDI se muestran en las secciones 2.1 y 2.4, y las dos páginas del tablero en la sección 4.3. Los archivos de las transformaciones están en etl/transformaciones/ del repositorio.